Business Case GoCase

Construção do Banco de Dados e Estruturação

Partes 5 e 6

Documentação, Justificativas e Requisitos Adicionais

Abril de 2026

Repositório: github.com/intelligent-life-paradox/business_case_gocase
Dashboard: businesscasegocase-jluca.streamlit.app
Dataset: kaggle.com - Olist Brazilian E-Commerce

Parte 5 — Documentação e Justificativas

1. Escolha e Origem dos Dados

Para responder ao desafio em tempo hábil preservando ao máximo os seus requisitos originais, optei por trabalhar com o dataset público **Olist Brazilian E-Commerce**, disponível via API do Kaggle:

kaggle.com/datasets/olistbr/brazilian-ecommerce

O dataset foi estudado diretamente na plataforma Kaggle e posteriormente normalizado no **Notebook 1** do repositório. A grande vantagem dessa escolha é que a maioria das entidades pedidas no desafio — produtos, pedidos, filiais, categorias e regiões — tem correspondência direta no schema da Olist, preservando a fidelidade do problema.

A única exceção foram os **dados de clientes**: por questões de privacidade, o dataset original não os disponibiliza com o nível de detalhe solicitado (nome, sexo, região estruturada). Esses registros foram portanto **injetados sinteticamente**, seguindo as distribuições demográficas e regionais realistas do Brasil, conforme descrito no Notebook 1.

2. Modelagem do Banco de Dados

Escolha do SQLite

A decisão de usar **SQLite** em vez de um SGBD cliente-servidor (PostgreSQL, MySQL) foi deliberada e motivada por **reprodutibilidade**: qualquer avaliador que clone o repositório e execute os notebooks chegará exatamente às mesmas conclusões, sem precisar configurar um servidor de banco de dados. O arquivo `varejo_gocase.db` é auto-contido e portátil.

Schema e Normalização

O banco segue a **3ª Forma Normal (3FN)**, com as seguintes tabelas principais:

- **Vendas** — tabela fato central, com chaves estrangeiras para todas as dimensões
- **Clientes** — dimensão com região e sexo
- **Filiais** — dimensão com estado e nome
- **Produtos** — dimensão com categoria e nome

Esse modelo em estrela simplificado foi suficiente para as 5 consultas analíticas exigidas, permitindo JOINs diretos sem subconsultas desnecessárias.

Escolha da Ferramenta de Visualização: Streamlit

Seguindo a proposta de inovar no problema, optei por usar **Streamlit** em vez do Power BI. Prefiro ferramentas *high-code* porque as acho mais manuseáveis — elas permitem explorar a criatividade com total controle sobre layout, lógica de filtragem, normalização de IDs e interatividade. O resultado é um dashboard com filtros globais (ano, sexo, região), explorador de filial e consulta individual de cliente, publicado em:

businesscasegocase-jluca.streamlit.app

Espero que gostem do resultado!

3. Lógica das Consultas SQL

Todas as consultas foram desenvolvidas com foco em **performance**, utilizando índices, planos de execução (`EXPLAIN QUERY PLAN`) e recursos avançados do SQL moderno.

Consulta 1 — Faturamento Mensal por Categoria

Objetivo: Calcular o faturamento total por mês e categoria no primeiro semestre de 2018.

Otimizações aplicadas:

- **Índice funcional**: criado em `strftime('%Y-%m', Data_Venda)` com `Valor_Total` e `ID_Produto` incluídos. Isso permite que o `GROUP BY` seja resolvido diretamente no índice, sem acessar a tabela fato.
- **Covering index**: os dados de soma são extraídos do próprio arquivo de índice, reduzindo I/O de disco a quase zero.
- O `EXPLAIN QUERY PLAN` foi executado e confirmou o uso do índice.

```sql
SELECT
    strftime('%Y-%m', v.Data_Venda) AS Mes,
    p.Categoria,
    ROUND(SUM(v.Valor_Total), 2) AS Faturamento_Total
FROM Vendas v
INNER JOIN Produtos p ON v.ID_Produto = p.ID_Produto
WHERE v.Data_Venda BETWEEN '2018-01-01' AND '2018-06-30'
GROUP BY 1, 2
ORDER BY 1 ASC, 3 DESC;
```

Listing 1: Consulta 1 — Faturamento por mês e categoria

Por que essa métrica importa? O faturamento por categoria e mês é a base para identificar sazonalidade, detectar categorias em declínio e planejar estoque com antecedência.

Consulta 2 — Produto Mais Vendido por Filial

Objetivo: Para cada filial, identificar o produto com maior volume vendido, ordenado por receita.

Otimizações aplicadas:

- **Composite covering index** em `(ID_Filial, ID_Produto, Quantidade, Valor_Total)`: o agrupamento e a soma são feitos sem ler a tabela fato principal.
- **Window function** `RANK() OVER (PARTITION BY ...)`: em vez de um JOIN complexo com subquery para encontrar o máximo, o SQLite calcula o ranking de todos os produtos dentro de cada filial de uma só vez, em uma única passagem.
- **CTE (Common Table Expression)**: organiza a lógica para que o filtro `WHERE Ranking = 1` seja aplicado sobre o conjunto já agregado, evitando dupla leitura.

```sql
WITH AgregadoFilial AS (
    SELECT
        f.Nome AS Filial,
        p.Nome AS Produto,
        SUM(v.Quantidade) AS Qtd_Total,
        SUM(v.Valor_Total) AS Receita_Total,
        RANK() OVER (
            PARTITION BY v.ID_Filial
            ORDER BY SUM(v.Quantidade) DESC
        ) AS Ranking
    FROM Vendas v
    INNER JOIN Filiais f ON v.ID_Filial = f.ID_Filial
    INNER JOIN Produtos p ON v.ID_Produto = p.ID_Produto
    GROUP BY v.ID_Filial, v.ID_Produto
)
SELECT Filial, Produto, Qtd_Total, ROUND(Receita_Total, 2)
FROM AgregadoFilial
WHERE Ranking = 1
ORDER BY Receita_Total DESC;
```

Listing 2: Consulta 2 — Produto mais vendido por filial com RANK()

Consulta 3 — Ticket Médio e LTV por Cliente

Objetivo: Calcular frequência de compras, ticket médio e valor total gasto (LTV) por cliente.

Otimizações aplicadas:

- **Covering index** em `(ID_Cliente, Valor_Total)`: o SQLite calcula `AVG` e `SUM` percorrendo apenas o índice, sem tocar na tabela `Vendas`.

```sql
SELECT
    ID_Cliente,
    COUNT(ID_Venda)           AS Frequencia_Compras,
    ROUND(AVG(Valor_Total),2) AS Ticket_Medio,
    ROUND(SUM(Valor_Total),2) AS Valor_Total_Gasto
FROM Vendas
GROUP BY ID_Cliente
ORDER BY Valor_Total_Gasto DESC
LIMIT 10;
```

Listing 3: Consulta 3 — Ticket médio e LTV

Por que essas métricas importam? Ticket médio e LTV são os dois indicadores centrais de valor de cliente. Juntos, permitem segmentar a base em estratégias de retenção e up-sell diferenciadas.

Consulta 4 — Clientes Fiéis por Região

Objetivo: Identificar clientes com mais de 3 compras, agrupados por região.

Otimizações aplicadas:

- **Filtro** `HAVING`: aplicado após o `GROUP BY` e otimizado pelo índice de `ID_Cliente`.
- **JOIN otimizado**: o SQLite usa o índice de PK de `Clientes` para o JOIN instantâneo sobre o conjunto já agregado.
- O critério de fidelidade (`X = 3`) foi parametrizado como variável, facilitando análises de sensibilidade.

```sql
SELECT
    c.Regiao,
    c.Nome AS Cliente,
    COUNT(v.ID_Venda) AS Total_Compras
FROM Vendas v
INNER JOIN Clientes c ON v.ID_Cliente = c.ID_Cliente
GROUP BY v.ID_Cliente
HAVING Total_Compras > 3
ORDER BY Total_Compras;
```

Listing 4: Consulta 4 — Clientes fiéis com HAVING

Consulta 5 — Tendências Regionais por Mês

Objetivo: Calcular variação percentual de vendas mês a mês por região, identificando crescimento ou queda.

Otimizações e decisões de design:

- **CTE duplo**: a primeira CTE (`VendasMensais`) agrega os dados; a segunda (`CalculoTendencia`) aplica a window function sobre o conjunto já reduzido, evitando múltiplas leituras da tabela fato.

- **Window function** `LAG()`: acessa o registro do mês anterior dentro de cada região sem releitura da tabela.
- **Filtro** `WHERE Venda_Mes_Anterior IS NOT NULL`: remove linhas com `NaN` que apareceriam no primeiro mês de cada região, garantindo um resultado limpo.
- **Filtro** `Mes_Sequencial <= 5`: garante que os primeiros meses de crescimento sejam apresentados de forma legível.

```sql
WITH VendasMensais AS (
    SELECT c.Regiao,
           strftime('%Y-%m', v.Data_Venda) AS Mes,
           SUM(v.Valor_Total) AS Venda_Atual
    FROM Vendas v
    INNER JOIN Clientes c ON v.ID_Cliente = c.ID_Cliente
    WHERE v.Data_Venda >= '2018-01-01'
    GROUP BY c.Regiao, Mes
),
CalculoTendencia AS (
    SELECT Regiao, Mes,
           ROUND(Venda_Atual, 2) AS Venda_Atual,
           ROUND(LAG(Venda_Atual) OVER (PARTITION BY Regiao ORDER BY Mes), 2)
               AS Venda_Mes_Anterior,
           ROUND(((Venda_Atual / LAG(Venda_Atual) OVER (
               PARTITION BY Regiao ORDER BY Mes)) - 1) * 100, 2)
               AS Crescimento_Pct,
           ROW_NUMBER() OVER (PARTITION BY Regiao ORDER BY Mes) AS Mes_Seq
    FROM VendasMensais
)
SELECT Regiao, Mes, Venda_Atual, Venda_Mes_Anterior, Crescimento_Pct
FROM CalculoTendencia
WHERE Venda_Mes_Anterior IS NOT NULL AND Mes_Seq <= 5
ORDER BY Regiao, Mes;
```

Listing 5: Consulta 5 — Tendência com LAG() e CTE duplo

Parte 6 — Requisitos Adicionais

1. Google Sheets com JavaScript — Gestão de Limites e Quotas

Sendo bem sincero, esse é um problema que eu nunca tive que lidar tanto na minha vida profissional como acadêmica. Pelo que pesquisei em várias fontes, minha resposta é a seguinte: para estruturar uma solução de extração via API com Google Sheets e JavaScript que evite problemas de quota, rate limiting e degradação de performance, adotaria a seguinte arquitetura:

Controle de Rate Limiting

Implementaria um mecanismo de **fila com back-off exponencial**: cada chamada de API seria enfileirada e, em caso de erro 429 (Too Many Requests) ou 503, o sistema aguardaria 2^n segundos antes de retentar, com um cap máximo (por exemplo, uns 64 segundos). Isso evita a chamada em loop desenfreado que consome toda a quota rapidamente - que é o que desejamos evitar.

Batching e Caching

Ao invés de fazer uma requisição por linha ou por célula, agruparia as chamadas em **lotes** (*batches*) usando a API Batch do serviço-alvo (ou `SpreadsheetApp.flush()` com escritas em bloco). Para dados que não mudam frequentemente, armazenaria os resultados em **cache** (ex: `CacheService` do Google Apps Script), com TTL definido pela frequência de atualização real dos dados, acredito que esse seja o padrão da indústria.

Controle de Quota por Projeto

Novamente, essa é uma questão que tive que pesquisar bastante e pensar para responder...

Usando como exemplo a Google cloud, eu separaria as extrações em **múltiplos projetos GCP** se necessário (ex: um projeto por domínio de dado), distribuindo a quota. Adicionaria um **log estruturado** de todas as chamadas (timestamp, endpoint, status) para monitorar consumo e antecipar gargalos.

Paralelismo Controlado

Usaria `Promise.allSettled()` com um semáforo de concorrência máxima (ex: 5 chamadas simultâneas), evitando que a extração paralela irrestrita estoure a quota em segundos.

2. Extrações SQL em Janelas Longas de Data

Lógica de Particionamento Temporal

Para janelas longas (ex: 3 anos de dados transacionais), nunca executaria uma query monolítica. Adotaria **particionamento incremental**: a extração processaria um intervalo fixo por vez (ex: 1-2 mêses), iterando sobre os períodos. Isso limita o uso de memória, permite retomar em caso de falha e facilita o paralelismo.

```sql
-- Executado para cada mes_inicio no intervalo desejado:
SELECT *
FROM Vendas
WHERE Data_Venda >= :mes_inicio
  AND Data_Venda <  :mes_inicio + INTERVAL '1 month'
  AND Data_Venda IS NOT NULL;
```

Listing 6: Estrutura de extração incremental por mês

Índices e Watermark

A tabela fato teria obrigatoriamente um índice em `Data_Venda` (e idealmente um índice de cobertura com as colunas mais consultadas). Para extrações recorrentes, manteria um **watermark** — um registro da última data processada com sucesso — e a próxima extração começaria dali, nunca reprocessando dados já capturados. Supondo esse cenário, é essencial sempre verificar as estatísticas das colunas mais lidas.

Menor Intervalo de Atualização Possível

O intervalo mínimo seria definido pela **latência de chegada dos dados na fonte**: se a operação gera dados com delay de 15 minutos (ex: processamento de pedidos), não há ganho em rodar a extração a cada 5 minutos — apenas geraria leituras vazias e carga desnecessária. A lógica seria: extrair somente quando `MAX(Data_Venda) na fonte > watermark armazenado`, evitando execuções nulas.

3. Estratégia de Consumo em Ambiente Analítico

A decisão entre conexão direta, camada intermediária e materialização depende de três variáveis: **volume**, **frequência de atualização** e **complexidade das transformações**.

Conexão Direta ao Banco

Adequada quando o volume é pequeno (até poucos milhões de linhas), as queries são simples e a frequência de acesso do BI é baixa. O risco é gerar carga no banco transacional em horário de pico — aceitável em ambientes analíticos isolados (ex: réplica de leitura).

Quando usar: protótipos, bases pequenas, relatórios ad-hoc ocasionais.

Carga em Camada Intermediária (Data Warehouse / Staging)

Quando o banco de origem é transacional e não pode ser sobrecarregado, os dados são extraídos via ETL para uma camada intermediária (ex: BigQuery, Redshift, DuckDB). O BI acessa essa camada, não o banco original. Permite aplicar regras de negócio na transformação e manter histórico.

Quando usar: volume médio a grande, múltiplas fontes, necessidade de histórico e auditoria.

Materialização para Consumo no BI

Quando as métricas são complexas (ex: LTV, coortes, rankings por janela deslizante) e recalculá-las a cada acesso seria custoso(!), opta-se por **materializar** os resultados — tabelas ou views materializadas atualizadas em schedule — e o BI lê diretamente a agregação pronta.

Quando usar: dashboards com alta frequência de acesso, métricas computacionalmente caras, SLA de resposta abaixo de 2 segundos.

Decisão Prática

No contexto deste case, usando SQLite + Streamlit, a conexão é direta ao banco com cache em memória (`@st.cache_data`), o que equivale funcionalmente a uma materialização leve: os dados são carregados uma vez e servidos em memória para todas as interações do usuário, sem reprocessamento a cada clique.